

Evaluation

A hands-on course in building agents by building their harness

Lesson 10

Every change is a bet

- Lesson 9's savings: a trim might cut the line that mattered, a summary might lose a fact
- Same for any change: a prompt line, a model, a step limit, a tool
- After the change the agent still runs and the trace still looks normal
- You find out the way you do for untested software: someone uses it, and it's wrong

Evaluation finds out first

Run the agent on tasks whose right outcome you already know, check what happened, and compare with before the change.

1. **Tasks with known outcomes**
2. **Grading**
3. **Comparing**

Grade the world, not the words

- A case is three things: a starting state, a task, a way to tell if the outcome is right
- The outcome is checked **without the agent**: a file, an exit code, the folder afterwards
- An agent that says "done" has proved nothing; a test that passes has
- Any route that ends in the right place passes, two steps or five
- Each case starts from a fresh folder, so cases can't affect each other

Code grades; two judges report beside it

- **By code:** a command whose exit status says pass or fail. Cheap, exact, the same every time
- But it checks exactly what you wrote, and can be passed without doing the job
- **By a model:** shown the task and the folder before and after, asked "was it done?"
- Two judges: a language model that answers in words, and **Jev**, which only answers typed questions

A grade you can't trust is worse than none, because you'll believe it.

A result means nothing alone

- The model isn't deterministic: the same case can pass once and fail the next time
- 11 of 12 is good or bad only against something: run the same cases **before and after**
- Record steps, tokens and time too: a change can pass everything and cost twice as much
- Exit non-zero when something got worse, so a git hook or CI job runs it on **every** change

Built on the whole harness, from the outside

- It treats the agent as a box: a task goes in, the world is changed
- That runs all five primitives together, so it catches what only shows when they work together
- **Control flow:** a loop that sets up, runs, grades, records, stops
- The task goes in as **input**; the check runs as **output**; the judges use the **model interface**
- Lesson 7's trace says what happened; evaluation says whether it was right

The concept: one case, two agents (abridged)

```
SETUP = "printf 'def add(a, b):\\n    return a - b\\n' > calc.py; printf ... > test.py"
INPUT = "test.py fails. Fix the bug in calc.py, not the test."
CHECK = "python3 test.py && grep -q 'add(2, 3)' test.py"
AGENTS = {"lesson 3": [sys.executable, LESSON_3, "agent", INPUT],
          "cheat": ["sed", "-i", "s/== 5/== -1/", "test.py"]}
```

- The check runs the test, and checks the assertion wasn't deleted
- `lesson 3` is Lesson 3's agent, started as a person would start it
- `cheat` is one `sed` I wrote: it changes what the test expects to -1

Two judges, one question (abridged)

```
def judges(state):  
    try: jev_says = round(jev.system_one(state, {"done": DONE})...["noul"], 2)  
    except Exception: jev_says = None  
    llm = client.messages.create(model="claude-sonnet-5-5", ..., messages=[...])  
    return jev_says, next(...).strip().upper().startswith("PASS")
```

- Same state for both: the request, the files before, the files after
- Never what the agent said; no answer from Jev is `None`, never a pass

The cheat passes the check

```
lesson 3  check pass  jev 0.98  llm pass  last time -  
cheat     check pass  jev 0.02  llm fail  last time -  
exit 0
```

- The test runs, the assertion is there: it just asserts the wrong thing
- Both judges failed it: they saw `test.py` before and after
- The check is still the grade: judges are models, and they get things wrong

quark: four cases, each checkable afterwards (abridged)

```
CASES = [  
    {"name": "count", "setup": "seq 1 37 > numbers.txt", "input": "How many lines are in numbers.txt? ...",  
     "check": "[ \"$(tr -d ' \\n' < answer.txt)\" = 37 ]"},  
    {"name": "fix", "setup": "printf ... > calc.py; printf ... > test.py", "input": "test.py fails. ...",  
     "check": "python3 test.py && grep -q 'add(2, 3)' test.py"},  
    {"name": "rename", "setup": "touch a.txt b.txt c.txt", "input": "Rename every .txt file ...",  
     "check": "[ -e a.md ] && [ -e b.md ] && [ -e c.md ] && ! ls *.txt 2>/dev/null"},  
    {"name": "remember", "setup": "true", "input": "Remember that I prefer short answers.",  
     "check": "grep -qi short .quark/memory/memory.md"},  
]
```

Run quark itself, in a fresh folder (abridged)

```
where = tempfile.mkdtemp(prefix=f"eval-{case['name']}-")
subprocess.run(case["setup"], shell=True, cwd=where)
seen, start = snapshot(where), time.time()
try: subprocess.run([sys.executable, os.path.abspath(__file__), case["input"]], cwd=where, ...)
except subprocess.TimeoutExpired: pass
passed = subprocess.run(case["check"], shell=True, cwd=where, ...).returncode == 0
jev_says, llm_says = judges(case["input"], seen, snapshot(where))
```

- Not a mock: the same file, prompt, guard, sandbox, retries, routing and tracing
- `input="y\n" * 50` answers the guard's questions; its policy still blocks
- Steps and tokens come from the child's trace; a failed case keeps its folder

The judges, in quark (abridged)

```
def judges(input, before, after):
    state = {"request": input, "files before": before, "files after": after}
    jev_says = ask(state, DONE)
    llm = call(max_tokens=1024, messages=[...])
    return jev_says and round(jev_says["noul"], 2), next(...).strip().upper().startswith("PASS")
```

- Jev through Lesson 5's `ask()` ; the language model through `call()` , the main model
- `snapshot()` : every file but `__pycache__` , plus `.quark/memory/` , cut to 2,000 characters
- Printed as `jev 0.97 llm pass ; JUDGES DISAGREE` if either differs from the check
- The grade, `REGRESSED` and the exit status still come from the check

It remembers last time

```
log, before = ".quark/evals.jsonl", {}  
os.makedirs(".quark", exist_ok=True)  
if os.path.exists(log):  
    for line in open(log): before[json.loads(line)["case"]] = json.loads(line)["passed"]  
cases, failed = [c for c in CASES if not names or c["name"] in names], 0
```

- A case that passed last time and fails now says **REGRESSED**
- Not "it failed", but "it used to pass"
- Run some cases by name: `quark.py --eval fix rename`

A baseline: what normal looks like

```
pass    count      2 steps    7.6s    26855 tokens   jev 0.97   llm pass
pass    fix         3 steps    9.1s    40469 tokens   jev 0.98   llm pass
pass    rename      2 steps    9.0s    26892 tokens   jev 0.97   llm pass
pass    remember    2 steps    8.1s    26969 tokens   jev 0.94   llm pass
4/4 passed
exit 0
```

- Both judges agree with every case
- About 27,000 to 40,000 tokens: the prompt is about thirteen thousand, read every step
- `fix` took a step more: it reads the code before changing it

Keep noise out of a judge's state

- The check runs `python3 test.py`, which leaves compiled bytes in `__pycache__`
- That's irrelevant detail, and it can throw Jev
- In testing, with it in the state, Jev was less sure of a right fix: 0.79 against 0.97
- So `snapshot()` skips any path with `__pycache__` in it

A change that looks harmless (output abridged)

`MAX_STEPS` from 20 to 1, to cap what a run can spend:

```
pass  count      1 steps    6.4s    13409 tokens  jev 0.97  llm pass
FAIL  fix         1 steps    6.1s    13381 tokens  jev 0.02  llm fail ...
pass  rename      1 steps    6.4s    13410 tokens  jev 0.97  llm pass
pass  remember    1 steps    6.4s    13455 tokens  jev 0.94  llm pass
3/4 passed
exit 1
```

- The `fix` line ends: `kept /tmp/eval-fix-d1y0lovx REGRESSED: it passed last time`
- `fix` can't be done in one step; the other three can
- Both judges agree with all four grades

The failed case kept its folder (shortened)

```
$ cat /tmp/eval-fix-d1y0lovx/calc.py
def add(a, b):
    return a - b
```

Its trace:

```
{"event": "start"}
{"event": "routed"}
{"event": "model", "stop_reason": "tool_use"}
{"event": "tool", "cmd": "cat calc.py test.py"}
{"event": "stopped"}
```

Other things we could do

- **Cases from reality:** every real mistake becomes a case, found in the traces
- **What passes:** final state, final answer, the path taken, or what must *not* happen
- **Noise and kinds:** pass at least once in k , or every time; regression cases apart from hard ones
- **When:** on every change, on a schedule, and on a sample of live traces
- **As a product:** Braintrust, LangSmith, promptfoo, Inspect run and grade cases, but can't give you the cases

Ideas worth knowing

- **Variants:** a cheaper model, another prompt, a lower limit, side by side against a baseline
- **Trials:** each case several times, a handful at once; read cost next to the pass rate
- **Test the grader:** answers whose grade you know, and store what the judge said
- **Judges' limits:** Jev doesn't count, and needs the folder before to see a change
- **An independent judge:** quark's language-model judge is the agent's own model

The rule

Write down what a right outcome looks like before you change anything, and then measure it after.

Evaluation runs the primitives, never rebuilds them

- **Control flow:** the loop under test is unchanged; the evaluation's loop is a separate one around it
- **Input:** the agent gets its task the way a person would give it
- **Context:** it doesn't edit the prompt, the memory or the working memory
- **Model interface:** the same calls; the judges go through `ask()` and `call()` afterwards
- **Output:** the tools run as always; the check is one more command beside them

Only the jobs you wrote down

- Four cases all passing is not a good agent; a set goes out of date
- It says something got worse, not why: that's the traces and the episode
- A few trials make noise look like signal
- A check can be passed without the job; a judge can be wrong
- None of this is solved by a bigger harness: keep reading failures, keep adding cases

Back to the course

Your own agent, its own cases, and the five primitives